Step 4 — Stream the Reply Live and Build One Structured Record per Ticket — Solution

> Back to this step · Hint 1 · Hint 2 · Solution

Story — Step 4: one reply is needed in two shapes. A support agent watching the screen wants to see it **as it's written**. The ticketing system wants a **checked record** — category, reply, fallback used, needs a human — to store and route on. Both must come from the same reply, and the unclear ticket and every fallback must be flagged for a person, not quietly decided. **If not:** the agent stares at a blank screen, and the ambiguity Step 1 revealed disappears without a trace.

After this step the project is complete: every reply streams live, and every ticket ends with a record that says whether a human should look at it.

Solution — a `TicketRecord`, a testable review rule, and a streaming helper

Changed from Step 3: `TicketRecord` adds two fields, both with defaults.

Story — `schemas.py`: `needs_human_review` is the flag a ticketing system routes on; `review_reasons` says **which** rule fired. **If not:** a person would see `True` and have to guess why.

```
# src/langchainpro/schemas.py
from typing import Literal

from pydantic import BaseModel

class TicketCategory(BaseModel):
    # why: the classifier's answer as a checked object, not free text —
    # only these 3 values can ever come back (Doc04's Literal)
    category: Literal["billing", "technical", "general"]
    # why: one short sentence, logged, so a wrong route can be explained
    reason: str
    # why: False when the ticket could fit more than one category — a
    # guess the rest of the pipeline should know about
    sure: bool

class TicketRecord(BaseModel):
    # why: one checked record per ticket — what a ticketing system would
```

```
    # store, count and route on (a machine reads this, not a person)
    customer_id: str
    ticket: str
    category: str
    reply: str
    # how: every field added in a later step has a default, so code
    # (and tests) written in an earlier step keep working unchanged
    used_fallback: bool = False
    # new in Step 4
    needs_human_review: bool = False
    # why: not just "yes, review it" — which rule fired, so a person
    # knows what to check first
    # new in Step 4
    review_reasons: list[str] = []
```

Changed from Step 3: `handle_ticket()` now streams the reply and fills the two new fields. Its name, inputs and return type stay the same.

Story — `services/pipeline.py`: `stream_reply()` prints each piece and returns the full text; `review_reasons()` holds the rules as plain code (fallback used, or not sure); `handle_ticket()` builds the record from the **same** streamed reply the customer saw. **If not:** a second model call for the record could quietly say something different from the reply.

```
# src/langchainpro/services/pipeline.py
# new in Step 4
from langchain_core.runnables import Runnable

from langchainpro.chains.classifier import classify_ticket
from langchainpro.chains.fallback import FALLBACK_REPLY
from langchainpro.chains.replies import build_reply_chain
from langchainpro.schemas import TicketCategory, TicketRecord

# new in Step 4
def stream_reply(chain: Runnable, inputs: dict) -> str:
    # why: a support agent watching the screen sees the reply as it is
    # written, instead of staring at nothing for several seconds
    # how: .stream() gives the reply in small pieces (Doc04's streaming,
    # through LCEL); print each piece, and keep them all for the record
    print("Reply: ", end="", flush=True)
    reply = ""
    for chunk in chain.stream(inputs):
        print(chunk, end="", flush=True)
        reply = reply + chunk
    print()
    return reply
```

```
# new in Step 4
def review_reasons(category: TicketCategory, used_fallback: bool) -> list[str]:
    # why: plain rules in plain code — easy to test, and a person can
    # see exactly which rule sent a ticket to them
    reasons = []
    if used_fallback:
        reasons.append("the fallback reply was sent")
    if not category.sure:
        reasons.append("the ticket could fit more than one category")
    return reasons

def handle_ticket(ticket: str, customer_id: str) -> TicketRecord:
    # how: classify -> pick the reply chain -> get the reply -> build
    # the record
    category = classify_ticket(ticket)
    chain = build_reply_chain(category.category)
    # how: every reply chain gets the same input dict; a prompt simply
    # ignores keys it doesn't use (Doc05's template edge cases)
    inputs = {"ticket": ticket, "customer_id": customer_id}
    # changed in Step 4: .stream(), not .invoke()
    reply = stream_reply(chain, inputs)

    # why: the fallback always returns exactly FALLBACK_REPLY, so an
    # equal reply means the fallback ran
    used_fallback = reply == FALLBACK_REPLY
    # new in Step 4
    reasons = review_reasons(category, used_fallback)

    # why: the record is built from the reply the customer actually saw
    # — never from a second model call that could say something else
    return TicketRecord(
        customer_id=customer_id,
        ticket=ticket,
        category=category.category,
        reply=reply,
        used_fallback=used_fallback,
        # new in Step 4
        needs_human_review=len(reasons) > 0,
        # new in Step 4
        review_reasons=reasons,
    )
```

Changed from Step 3: `print_record()` adds the review flag and its reasons; the reply itself was already streamed.

Story — **`main.py`** **:** the loop only shows results; the review reasons are listed under each ticket. **If not:** a flagged ticket would look the same as a clean one.

```
# src/langchainpro/main.py
import json
import sys

import openai

from langchainpro.config import load_config
from langchainpro.exceptions import MissingConfigError
from langchainpro.schemas import TicketRecord
from langchainpro.services.pipeline import handle_ticket
from langchainpro.utils.logger import get_logger

# how: a path from the repo root — run every command from there
SAMPLES_PATH = "data/sample/tickets.json"

def load_samples() -> list[dict]:
    # why: the same 4 tickets every run, shared with the tests (Doc02
    # taught reading JSON)
    with open(SAMPLES_PATH) as samples_file:
        # how: read the file as text, then json.loads() it (Doc02)
        return json.loads(samples_file.read())

def print_record(record: TicketRecord) -> None:
    print(f"  category:      {record.category}")
    print(f"  used fallback: {record.used_fallback}")
    # new in Step 4
    print(f"  human review:  {record.needs_human_review}")
    # new in Step 4
    for reason in record.review_reasons:
        print(f"    - {reason}")

def main() -> None:
    # why: config first — a missing OPENAI_API_KEY stops the app here
    try:
        load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)

    logger = get_logger(__name__)
    samples = load_samples()
    print(f"LangChainPro — {len(samples)} tickets in the queue.")

    for sample in samples:
        print()
        print(f"Ticket from {sample['customer_id']}: {sample['ticket']}")
        try:
```

```
            record = handle_ticket(sample["ticket"], sample["customer_id"])
        except openai.AuthenticationError:
            # why: a bad key is permanent — no retry fixes it, so exit
            logger.error("Authentication failed — check OPENAI_API_KEY.")
            print("Login failed — check OPENAI_API_KEY in your .env file.")
            sys.exit(1)
        print_record(record)

if __name__ == "__main__":
    main()
```

Story — the tests: `test_review.py` checks each review rule with hand-made inputs, no key needed. `test_schemas.py` is unchanged: its Step 1 record still builds, because the new fields have defaults. The integration test now checks the unclear ticket really gets flagged. **If not:** a rule could stop firing and nobody would notice.

```
# tests/unit/test_review.py
# Checks the "needs human review" rules with hand-made inputs.
# No API key needed. Run from the repo root:
#   python tests/unit/test_review.py
from langchainpro.schemas import TicketCategory
from langchainpro.services.pipeline import review_reasons

def make_category(sure: bool) -> TicketCategory:
    return TicketCategory(category="billing", reason="test", sure=sure)

def test_clear_ticket() -> None:
    reasons = review_reasons(make_category(True), False)
    print(f"case 1 (sure, no fallback) -> {reasons}")

def test_unsure_ticket() -> None:
    reasons = review_reasons(make_category(False), False)
    print(f"case 2 (unsure)            -> {reasons}")

def test_fallback_used() -> None:
    reasons = review_reasons(make_category(True), True)
    print(f"case 3 (fallback)          -> {reasons}")

if __name__ == "__main__":
    test_clear_ticket()
    test_unsure_ticket()
    test_fallback_used()
```

Changed from Step 1: the integration test checks the review flag on a clear and an unclear ticket.

```
# tests/integration/test_live_pipeline.py
# Runs the classifier and the whole pipeline against the real API.
# Makes real API calls — needs a valid OPENAI_API_KEY in .env.
# Run from the repo root: python tests/integration/test_live_pipeline.py
import json

from langchainpro.chains.classifier import classify_ticket
from langchainpro.config import load_config
from langchainpro.services.pipeline import handle_ticket

def load_samples() -> list[dict]:
    with open("data/sample/tickets.json") as samples_file:
        # how: read the file as text, then json.loads() it (Doc02)
        return json.loads(samples_file.read())

def test_classification(samples: list[dict]) -> None:
    # case 1: every sample ticket gets the category we expect
    for sample in samples:
        result = classify_ticket(sample["ticket"])
        print(f"case 1 (expected {sample['expected']}) -> "
              f"{result.category}, sure={result.sure}")

def test_full_pipeline(samples: list[dict]) -> None:
    # case 2: the billing ticket goes all the way to a reply
    billing = samples[0]
    record = handle_ticket(billing["ticket"], billing["customer_id"])
    print(f"case 2 (billing)  -> {record.category}")
    # new in Step 4
    print(f"                     review={record.needs_human_review}")

    # new in Step 4
    # case 3: the unclear ticket — must be flagged for a human
    unclear = samples[3]
    record = handle_ticket(unclear["ticket"], unclear["customer_id"])
    print(f"case 3 (unclear)  -> review={record.needs_human_review}, "
          f"{record.review_reasons}")

if __name__ == "__main__":
    load_config()
    samples = load_samples()
    test_classification(samples)
    test_full_pipeline(samples)
```

Run it:

```
python tests/unit/test_review.py
python tests/unit/test_schemas.py
python tests/integration/test_live_pipeline.py
python -m langchainpro.main
```

`test_review.py` prints:

```
case 1 (sure, no fallback) -> []
case 2 (unsure)            -> ['the ticket could fit more than one category']
case 3 (fallback)          -> ['the fallback reply was sent']
```

`python -m langchainpro.main` ends each ticket like this (log lines left out; model replies vary):

```
Ticket from C-1004: My invoice and the dashboard numbers don't match. Bug?
Reply: Sorry about the mismatch. Our billing team will check your
invoice within 2 business days.
  category:      billing
  used fallback: False
  human review:  True
    - the ticket could fit more than one category
```